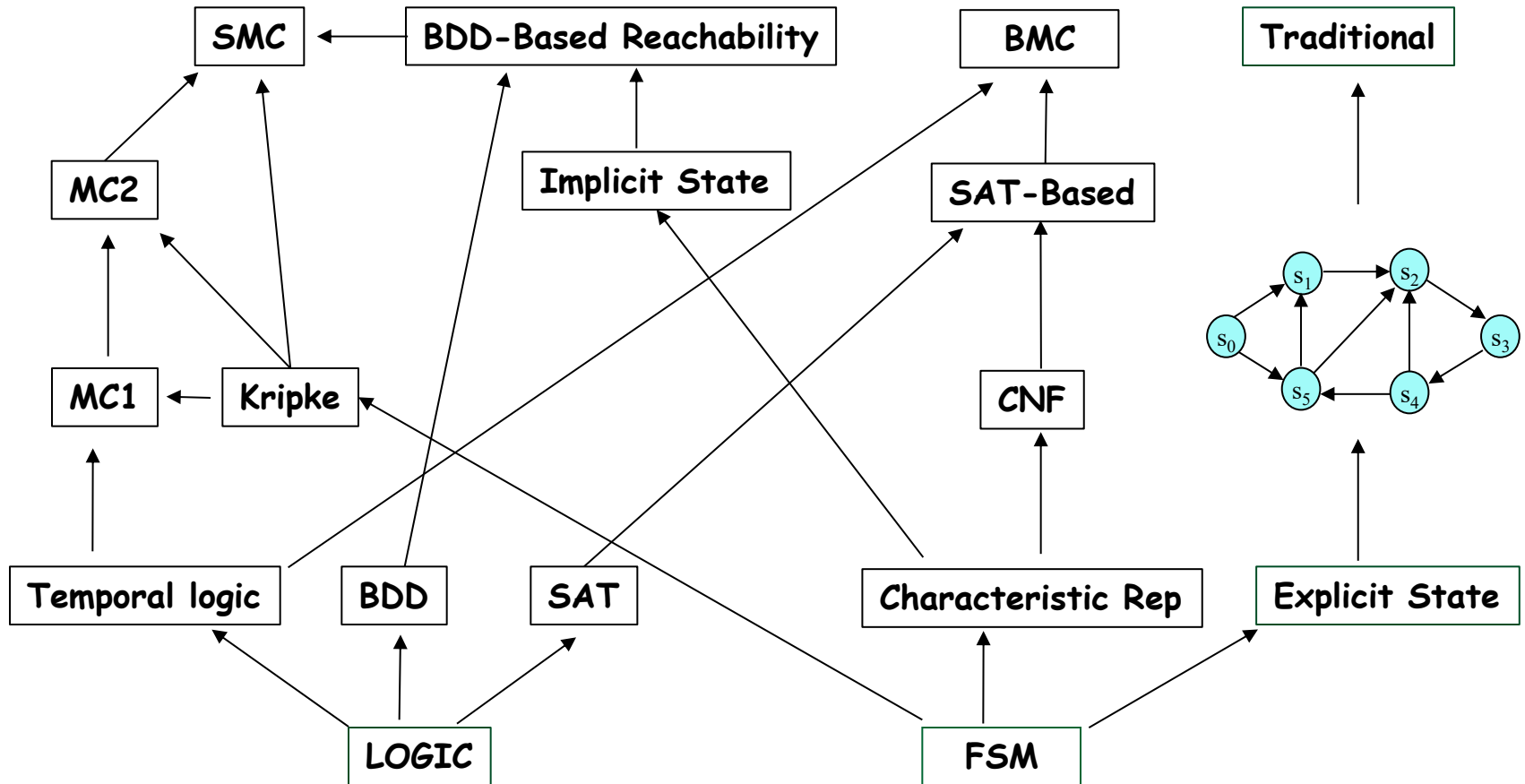




Preliminaries: Review



A Road Map





Boolean Algebra



Boolean Algebra

- XOR : $\oplus$
 - $x \oplus y = x y' + x' y$
- $A_1 \oplus A_2 \oplus \dots \oplus A_n = 1$ if and only if the number of 1s is odd.
- XNOR gate (sometimes NXOR): $\otimes$
 - $x \otimes y = x y + x' y' = (x \oplus y)'$

Boolean Algebra

- Redundancy Law:

- $A + A' B = A + B$

- Consensus Law:

- $Ax + A'y + xyB = Ax + A'y$

Factorization

- $M = a \oplus b = (a+b)(a'+b')$
- $M' = (a \oplus b)' = (a'+b)(a+b')$
- $ABC+X+Y+Z$
 - $= ABC+(X+Y+Z)$
 - $= (A+X+Y+Z) (B+X+Y+Z) (C+X+Y+Z)$

Boolean Operators

- Residual function (cofactor):

$$f \mid_{x_i \leftarrow b} (x_1, \dots, x_n) = f(x_1, \dots, x_{i-1}, b, x_{i+1}, \dots, x_n)$$

- Given a function $f(x_1, x_2, x_3)$
 - Existential quantification of f w.r.t. x_1 is
 - $\exists x_1. f(x_1, x_2, x_3) = f(0, x_2, x_3) + f(1, x_2, x_3)$
 - Universal quantification of f w.r.t. x_1 is
 - $\forall x_1. f(x_1, x_2, x_3) = f(0, x_2, x_3) \& f(1, x_2, x_3)$
 - Unique quantification (Boolean Difference) of f w.r.t. x_1 is
 - $!x_1. f(x_1, x_2, x_3) = df/dx_1 = f(0, x_2, x_3) \oplus f(1, x_2, x_3)$

Shannon's Decomposition

$$Y = f(x_1, x_2, \dots, x_n) = x_i f_{x_i} + \bar{x}_i f_{\bar{x}_i}$$

where

$$f_{x_i=1} = f_{x_i} = f(x_1, x_2, \dots, x_{i-1}, 1, x_{i+1}, \dots, x_n)$$

$$f_{x_i=0} = f_{\bar{x}_i} = f(x_1, x_2, \dots, x_{i-1}, 0, x_{i+1}, \dots, x_n)$$

Notations for Operators

■ Logic

- AND: $a \wedge b$
- OR: $a \vee b$
- NEGATION: $\neg a$ (a')

■ Digital Systems

- ab
- $a+b$
- $\overline{a}$

- Notation a' is used for negation of a whenever no next state variables are involved.



Set Theory



Set Theory

- A set is an unordered collection of objects referred to as elements.
- A set is said to contain its elements.

Sets

- A set is a well defined collection of distinct objects.
- The objects that make up a set (also known as the elements or members of a set) can be anything:
 - numbers, people, letters of the alphabet, other sets, and so on.
- Example :
 - $A = \{a,b,c\}$ $B = \{1,2\}$

Set Representations

- List Notation :
 - Listing each member of the set. $Q = \{\text{blue, green, red}\}$
- Predicate Notation :
 - We can specify the members of a set by specifying a condition that they all meet.
 - $N = \{1, 2, 3, 4, \dots\} = \{x \mid x \text{ is a natural number}\}$
 - $E = \{2, 4, 6, 8, \dots\} = \{2x \mid x \text{ is a natural number}\}$
 - The vertical bar (" $\mid$ ") means "such that",
 - E is the set of all numbers of the form $2x$, such that x is a natural number.

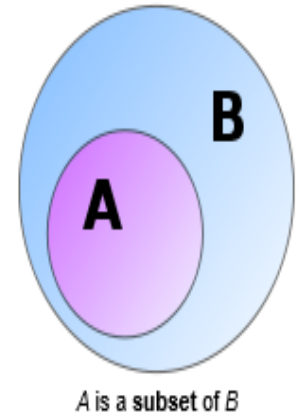
List Notation

- Explicitly:
 - listing the elements of a set
 - $\{1, 1, 2, 3, 3\} = \{1, 2, 3\}$ since repetition is irrelevant.
 - $\{1, 2, 3\} = \{3, 2, 1\}$ since sets are unordered.
 - $\{1, 2, 3, \dots, 99\}$ is the set of positive integers less than 100; use ellipses when the general pattern of the elements is obvious.
 - $\{1, 2, 3, \dots\}$ is a way we denote an infinite set (the natural numbers).
 - $\emptyset = \{\}$ is the empty set, or the set containing no elements.

Predicate Notation

- Implicitly:
 - by using a set builder notations, stating the property or properties of the elements of the set.
 - $S = \{m \mid 2 \leq m \leq 100, m \text{ is integer}\}$
 - S is the set of all m such that m is between 2 and 100 and m is integer.

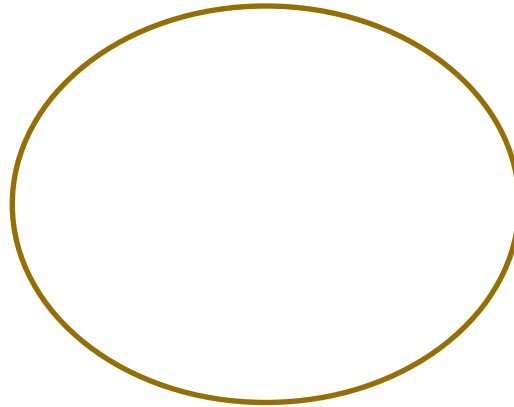
Subsets



- If every member of set A is also a member of set B ,
 - then A is said to be a subset of B , written $A \subseteq B$.
 - Equivalently, we can write $B \supseteq A$, read as B is a superset of A .
- The empty set is a subset of every set and every set is a subset of itself:
 - $\emptyset \subseteq A$.
 - $A \subseteq A$.
- Consider a set $A = \{a, b, c\}$.
 - $\emptyset, \{a\}, \{b\}, \{c\}, \{a, b\}, \{b, c\}, \{a, c\}, \{a, b, c\}$ are the subsets of A .

Empty Set

$A =$

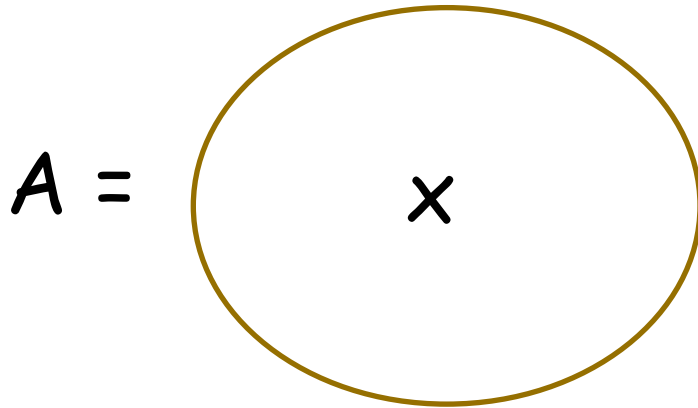


$$A = \{ \} = \emptyset$$

The Power Set: 2^A

- The power set of a set A is the set of all subsets of A , including A itself and the empty set.
- Consider a set $A = \{a, b, c\}$.
 - $2^A = \{ \emptyset, \{a\}, \{b\}, \{c\}, \{a, b\}, \{b, c\}, \{a, c\}, \{a, b, c\} \}$
- The power set of a finite set with n elements has 2^n elements.
- The size of a power set is represented as, $|2^A| = 2^{|A|}$

Characteristic Function of Sets



$$f_A(x) = 1 \text{ if } x \in A$$

$$f_A(x) = 0 \text{ if } x \notin A$$

- Sets $\rightarrow$ Boolean functions

Logic and Set

■ Logic

- AND: $a \wedge b$
- OR: $a \vee b$
- NEGATION: $\neg a$ (a')

■ Set

- Intersection: $a \cap b$
- Union: $a \cup b$
- Complement: $\sim a$

Set Operations

- Element-based Approach
- $A \cap B$ means $\forall x (x \in A \cap B) \text{ implies } (x \in A) \wedge (x \in B)$.
- $A \cup B$ means $\forall x (x \in A \cup B) \text{ implies } (x \in A) \vee (x \in B)$.
- $A \subseteq B$ means "A is a subset of B."
- $A \subseteq B$ iff $\forall x (x \in A) \text{ implies } (x \in B)$.
- $A = B$ iff A and B have exactly the same elements.
- $A = B$
 - iff $A \subseteq B$ and $B \subseteq A$
 - iff $\forall x ((x \in A) \text{ implies } (x \in B)) \text{ and } \forall x ((x \in B) \text{ implies } (x \in A))$

A - B

- The difference between the two sets, A and B , written as $A \setminus B$ or $A - B$, is a set that contains those elements of A that are NOT in B .
- To find the difference, we remove all the elements of set B from set A .
- $A - B = \{x \mid x \in A \text{ and } x \notin B\}$

Element-based Approach

- $A \cap (B \cup C) = (A \cap B) \cup (A \cap C)$
- **Proof** Let x be arbitrary.
 - $(\subseteq)$
Assume $x \in A \cap (B \cup C)$.
Then $x \in A$ and $x \in B \cup C \rightarrow x \in B$ or $x \in C$.
 - Case 1: if $x \in B$, then $x \in A \cap B$.
 - Case 2: if $x \in C$, then $x \in A \cap C$.
 - Thus $x \in (A \cap B) \cup (A \cap C)$.
 - $(\supseteq)$
Assume $x \in (A \cap B) \cup (A \cap C)$.
Then either:
 - $x \in A \cap B$, so $x \in A$ and $x \in B \subseteq B \cup C$, or
 - $x \in A \cap C$, so $x \in A$ and $x \in C \subseteq B \cup C$.
 - Thus $x \in A \cap (B \cup C)$.

- $A - (B \cap C) = (A - B) \cup (A - C)$
- **Proof** Let x be arbitrary.
- $\subseteq$
 - Assume $x \in A - (B \cap C)$. Then
 - $x \in A$,
 - $x \notin B \cap C$, meaning **at least one of** $x \notin B$ or $x \notin C$.
 - Case 1: If $x \notin B$, then $x \in A - B$.
 - Case 2: If $x \notin C$, then $x \in A - C$.
 - Thus $x \in (A - B) \cup (A - C)$.
- $\supseteq$
 - Assume $x \in (A - B) \cup (A - C)$.
Two cases:
 - If $x \in A - B$, then $x \in A$ and $x \notin B$.
So $x \notin B \cap C$, hence $x \in A - (B \cap C)$.
 - If $x \in A - C$, similarly $x \in A$ and $x \notin C$,
so $x \in A - (B \cap C)$.

- Exercise: If $A \subseteq B$, then, for any C , we have $A \cap C \subseteq B \cap C$.

Cartesian Product

- An ordered pair is a sequence of two elements, a and β . We write these elements between angled brackets, $\langle a, \beta \rangle$.
- There are crucial differences between ordered pairs and sets.

- Ordered Pairs

- $\langle a, b \rangle \neq \langle b, a \rangle$

- $\langle b, b \rangle \neq \langle b \rangle$

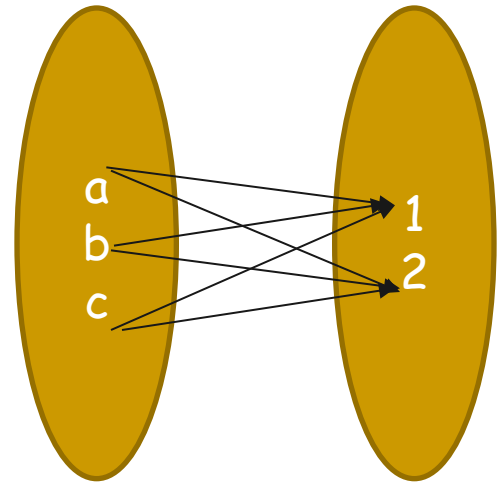
- Sets

- $\{a, b\} = \{b, a\}$

- $\{b, b\} = \{b\}$

Cartesian Product

- Suppose we have two sets $A = \{a, b, c\}$ and $B = \{1, 2\}$.
- We form ordered pairs by taking an element of A as the first member of the pair and an element of B as the second member.
- The set containing all such ordered pairs is called the
- Cartesian Product of A and B , written $A \times B$.
- $A \times B = \{ \langle a, \beta \rangle \mid a \in A \text{ and } \beta \in B \}$
- $A \times B = \{ \langle a, 1 \rangle, \langle a, 2 \rangle, \langle b, 1 \rangle, \langle b, 2 \rangle, \langle c, 1 \rangle, \langle c, 2 \rangle \}$

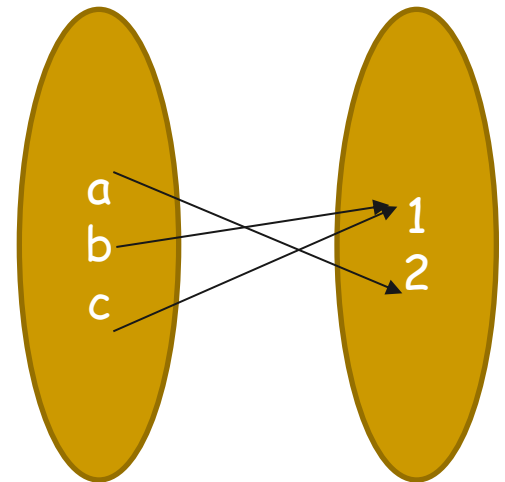


$A \times B$

- $A \times B = \{ \langle a, \beta \rangle \mid a \in A \text{ and } \beta \in B \}$
- $A \times B = \{ \langle a, 1 \rangle, \langle a, 2 \rangle, \langle b, 1 \rangle, \langle b, 2 \rangle, \langle c, 1 \rangle, \langle c, 2 \rangle \}$
- $A \times B$ means
- $\forall x (x \in A \times B) \rightarrow \exists m \exists n. (m \in A) \wedge (n \in B) \wedge (x = \langle m, n \rangle \in A \times B)$

Relation

- A relation is a collection of ordered pairs of elements.
- It is a subset of the Cartesian product.
- A relation between two sets A and B is a subset of $A \times B$.
- For example: consider $A = \{a, b, c\}$ and $B = \{1, 2\}$ then a relation R can be defined as $R = \{ \langle a, 2 \rangle, \langle b, 1 \rangle, \langle c, 1 \rangle \}$ which is a subset of the Cartesian product $A \times B$.

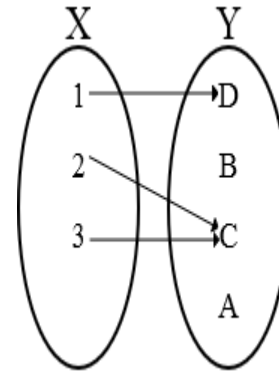


Function

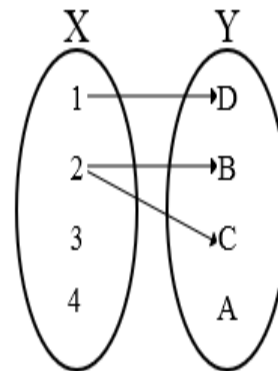
- A function is a special type of relation.
- It is a relation with a special restriction on the first coordinate.
- A function f with domain X and co-domain Y is commonly denoted by $f : X \rightarrow Y$
- For each argument x , the corresponding unique y in the co-domain is called the function value at x : $y = f(x)$

Function

- A function

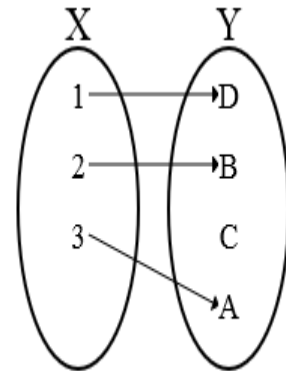


- Not a function



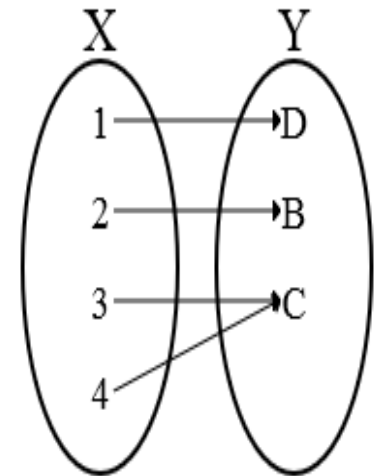
Injective or One-to-one Function

- An injective function or injection or one-to-one function is a function that preserves distinctness:
 - it never maps distinct elements of its domain(X) to the same element of its codomain(Y).
 - $f(a) \neq f(b)$ for any two different elements a and b of the domain X .



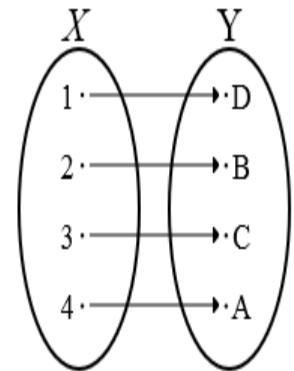
Surjective or Onto Function

- It is Surjective if for every element y in the codomain there is an x in the domain such that $f(x) = y$.
- A function f from a set X to a set Y is surjective (or onto), or a surjection,
 - if every element y in Y has a corresponding element x in X such that $f(x) = y$.
- The function f may map more than one element of X to the same element of Y .
- Example for a Surjective Function and not an Injective function is shown below.



Bijjective Function (One-to-one Correspondence)

- If it is both injective and surjective, then it is called a Bijjective Function.
- A bijection (or bijective function or one-to-one correspondence) is a function between the elements of two sets.
- Every element of one set is paired with exactly one element of the other set, and every element of the other set is paired with exactly one element of the first set.
- There are no unpaired elements.

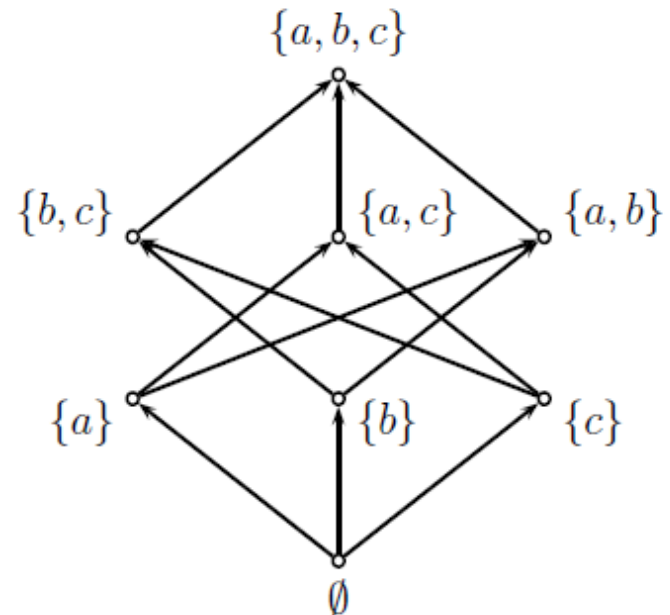


A Partial Order

- A binary relation, $\leq$, on a set, X , is a partial order
 - (Reflexivity): $a \leq a$, for all $a \in X$;
 - (Transitivity): If $a \leq b$ and $b \leq c$, then $a \leq c$, for all $a, b, c \in X$.
 - (Anti-symmetry): If $a \leq b$ and $b \leq a$, then $a = b$, for all $a, b \in X$.

Examples of Partial Orders

- The subset ordering.
 - $\subseteq$, on a set, X , is a partial order.



A Total Order

- A partial order is a total order iff for all $a, b \in X$, either $a \leq b$ or $b \leq a$.



Recursion



Iterative Computing: $f(5) = 5!$

- $f(5) = 5*4*3*2*1$

- 1

- $(1)*2$

- $(1*2)*3$

- $(1*2*3)*4$

- $(1*2*3*4)*5$

- $f(5) = 5*4*3*2*1$

- 5

- $(5)*4$

- $(5*4)*3$

- $(5*4*3)*2$

- $(5*4*3*2)*1$

Recursion

- A recursive method is any method that **calls itself**
 - Each call solves an identical problem
 - The code is the same!
 - Successive calls solve smaller/simpler instances
 - Every recursive algorithm has at least one base case
- **Base case**
 - VERY IMPORTANT
 - Stops the recursion (prevents infinite loops)
 - Solved directly to return a value without calling the same method again

Recursion: $f(5) = 5!$

- Stage 1: Sink

- $f(n) = n * f(n-1)$

- $f(5) = 5 * f(4)$

- $f(4) = 4 * f(3)$

- $f(3) = 3 * f(2)$

- $f(2) = 2 * f(1)$

- $f(1) = 1$



- Stage 2: Return

- $f(n) = n * f(n-1)$

- $f(5) = 5 * f(4)$

- $f(4) = 4 * f(3)$

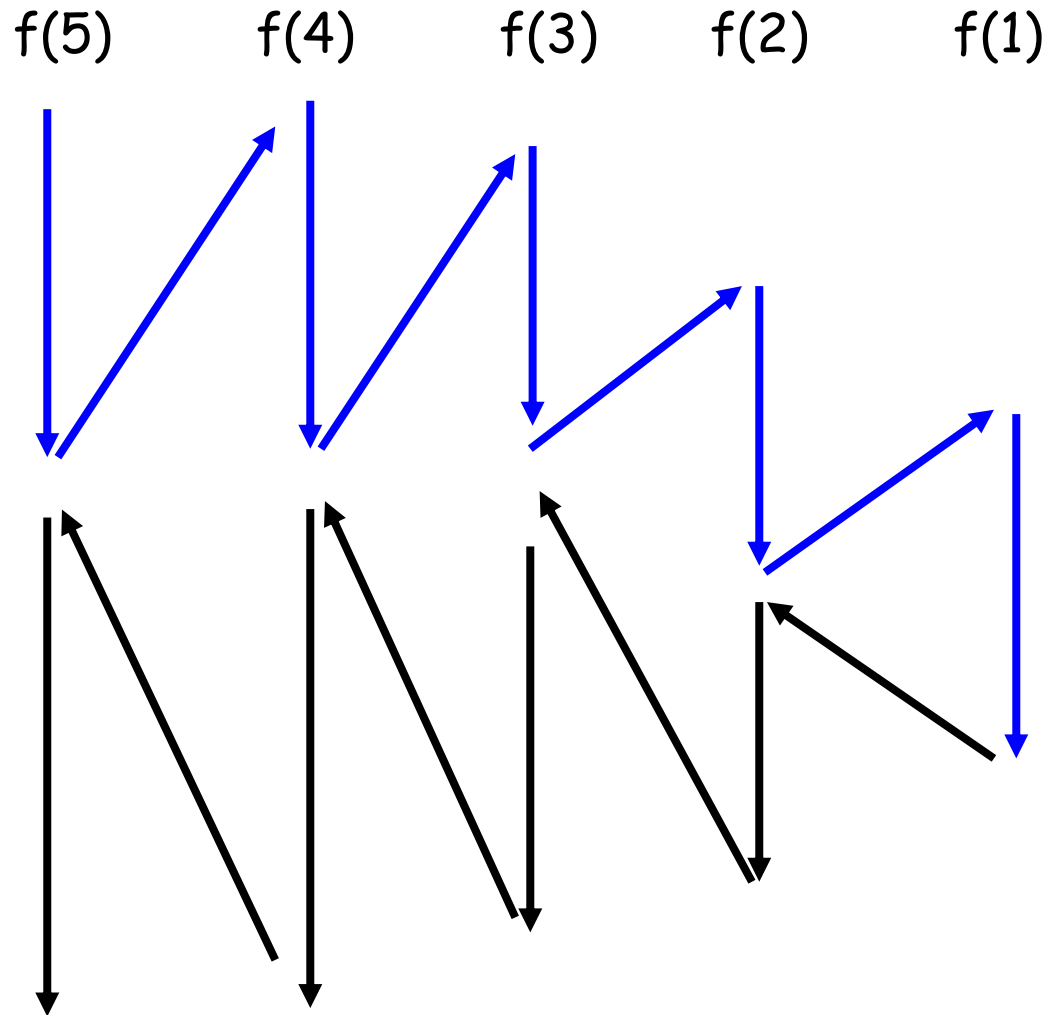
- $f(3) = 3 * f(2)$

- $f(2) = 2 * f(1)$

- $f(1) = 1$



Recursion: $f(5) = 5!$



■ Stage 1: Sink

■ $f(n) = n * f(n-1)$

□ $f(5) = 5 * f(4)$

□ $f(4) = 4 * f(3)$

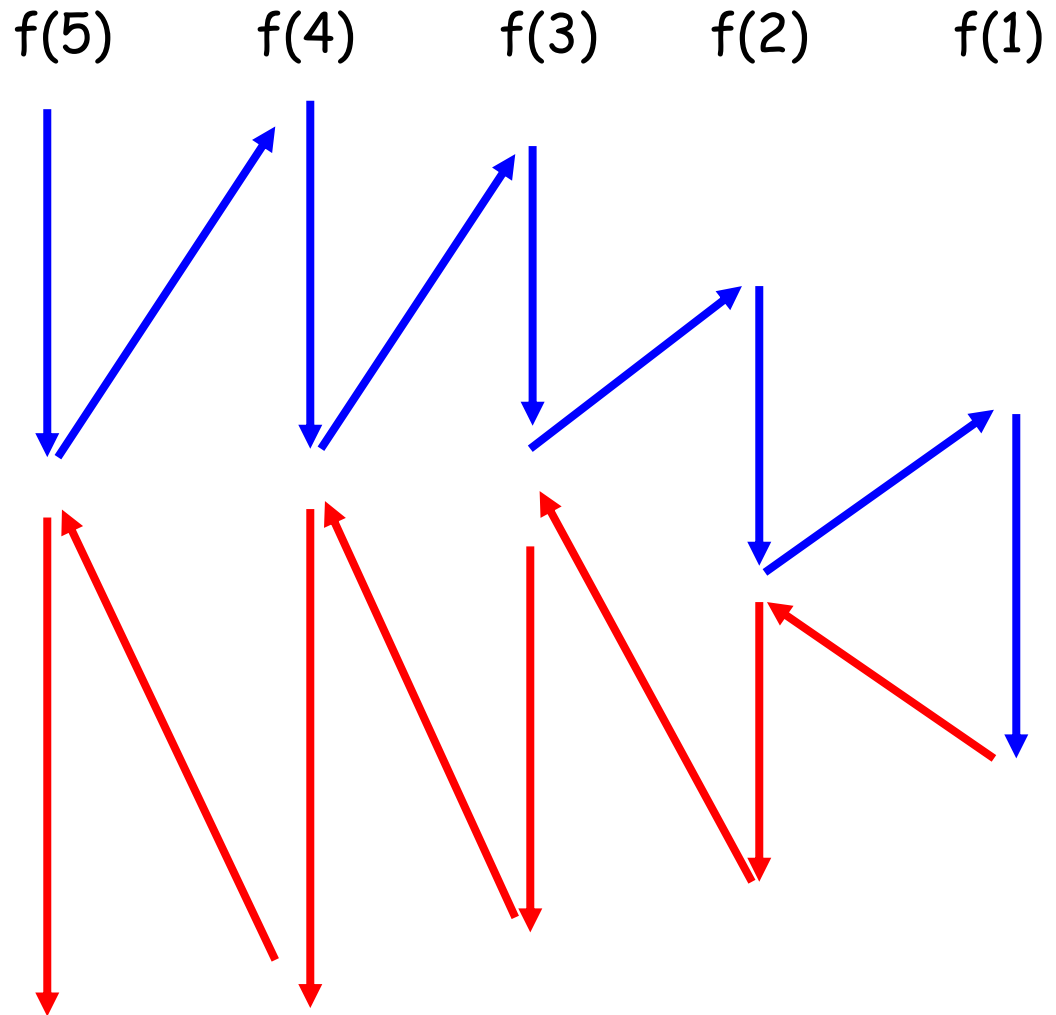
□ $f(3) = 3 * f(2)$

□ $f(2) = 2 * f(1)$

□ $f(1) = 1$



Recursion: $f(5) = 5!$



■ Stage 2: Return

■ $f(n) = n * f(n-1)$

□ $f(5) = 5 * f(4)$

□ $f(4) = 4 * f(3)$

□ $f(3) = 3 * f(2)$

□ $f(2) = 2 * f(1)$

□ $f(1) = 1$



C Program for Factorial with Recursion

```
① #include <stdio.h>
② int main() {
③     int x = 7;
④     printf("The factorial of the number is %d", fact(x));
⑤     return 0;
⑥ }
⑦ // Recursive function to find factorial
⑧ int fact(int y) {
⑨     if (y == 0)
⑩         return 1;
⑪     return y * fact(y - 1);
⑫ }
```

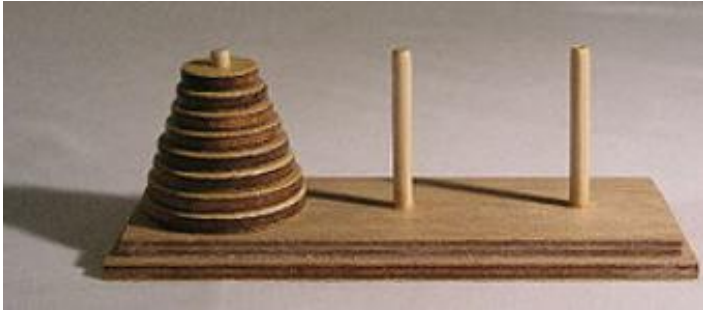
Recursion or Iteration?

- There is usually more than one way to solve a problem!
 - Iteration (loops to repeat code)
 - Recursion (nested function calls to repeat code)
- Tradeoffs between two options:
 - Sometimes recursive solution is easier
 - Recursive solution is often slower

Recursion or Iteration?

- Overhead involved with recursion
 - Memory space
 - Time of execution
- Today's computers
 - fast
 - inexpensive memory
- If an iterative solution is more obvious and easier to understand than a recursive solution
 - Use the iterative solution, which would be more efficient.
- When the recursive solution is more obvious or easier to construct ... use recursion

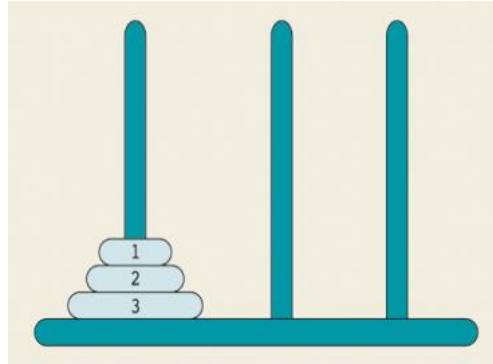
Towers of Hanoi



- The legend of the temple of Brahma
 - 64 golden disks on 3 pegs
 - The universe will end when the priest move all disks from the first peg to the last

- Rules:
 - Only move one disk at a time
 - A move is taking one disk from a peg and putting it on another peg (on top of any other disks)
 - Cannot put a larger disk on top of a smaller disk
 - *With 64 disks, at 1 second per disk, this would take roughly 585 billion years*

Towers of Hanoi: Three Disk Problem



```
1. solveTowers (count, source, destination, spare) {  
2.   if (count == 1) { move directly }  
3.   else {  
4.       solveTowers(count-1, source, spare, destination)  
5.       solveTowers(1, source, destination, spare)  
6.       solveTowers(count-1, spare, destination, source)  
7.   }  
8. }
```



Quantified Statements



Quantified Statements

- The words "all" "some" and "none" are examples of quantifiers.
- A statement containing one or more of these words is a quantified statement.
- Quantifiers give the 'range' and 'universe' over which the statement is being claimed.

Universal Quantifier: $\forall$

- A universal quantifier states that an entire set of things share a characteristic.
- Universal Quantifier:
 - A few ways to say $\forall x \in \mathbb{N}$:
 - "For all natural numbers $x, \dots$ ",
 - "For all $x \in \mathbb{N}, \dots$ ",
 - "For any $x \in \mathbb{N}, \dots$ ",
 - "For every $x \in \mathbb{N}, \dots$ "

Existential Quantifier: $\exists$

- An existential quantifier states that a set contains at least one element.
- Existential Quantifier:
 - A few ways to say $\exists x \in \mathbb{N}$:
 - "There exists a natural number x such that ...",
 - "There exists $x \in \mathbb{N}$ such that ...",
 - "For at least one $x \in \mathbb{N}$...",
 - "For some $x \in \mathbb{N}$..."

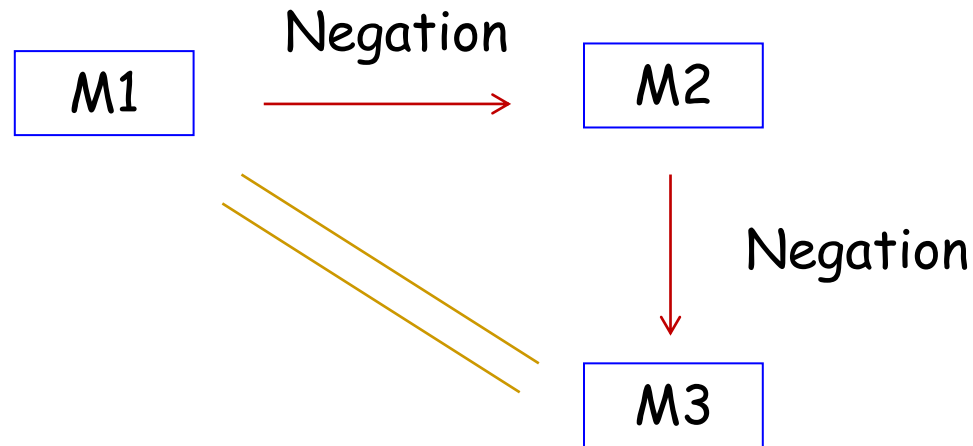
Negations of Quantified Statements

- Statement: All A are B.
- Negation: Some A are not B.
- Example:
 - All people take exams honestly.
- Negation:
 - Some people do not take exams honestly.

Negations of Quantified Statements

- Statement: Some A are B.
- Negation: No A are B.
- Example:
 - Some roads are open.
- Negation:
 - No roads are open.

Negations of Quantified Statements



■ Example 1:

- The negation of "all A are B " is "at least one A is not B ".
- The negation of "at least one A is not B " is "all A are B ".

■ Example 2:

- The negation of "no A are B " is "at least one A is B ".
- The negation of "at least one A is B " is "no A are B ".

What is a Predicate?

- A **predicate** is a function that takes one or more variables and **returns a truth value**:
 - **True**
 - **False**
- Formally, in logic:
 - $P: D \rightarrow \{true, false\}$
 - D = domain of discourse (set of objects we are talking about)
- Example: $P(x)$ = “ x is even”
- Domain: integers
 - For $x = 4$, $P(x)$ = true
 - For $x = 5$, $P(x)$ = false

Negation of Quantified Statements

Original Statement	Negation	Notes
$\forall x, P(x)$	$\exists x, \neg P(x)$	"For all" becomes "there exists" and predicate is negated
$\exists x, P(x)$	$\forall x, \neg P(x)$	"There exists" becomes "for all" and predicate is negated
$P \rightarrow Q$	$P \wedge \neg Q$	Important for conditional statements
$P \wedge Q$	$\neg P \vee \neg Q$	De Morgan's Law
$P \vee Q$	$\neg P \wedge \neg Q$	De Morgan's Law

- **Statement:** $\forall x \in \mathbb{N}, x^2 \geq 0$
- **Negation Rule:**
- $\neg(\forall x, P(x)) \equiv \exists x, \neg P(x)$
 $\neg(\forall x \in \mathbb{N}, x^2 \geq 0) \equiv \exists x \in \mathbb{N} \text{ such that } x^2 < 0$
-  Simplified in words:
"There exists a natural number whose square is negative."
- **Comment:** This statement is **false**, because squares of natural numbers are always ≥ 0 .

Negation of Quantified Statements

Original Statement	Negation	Notes
$\forall x, P(x)$	$\exists x, \neg P(x)$	"For all" becomes "there exists" and predicate is negated
$\exists x, P(x)$	$\forall x, \neg P(x)$	"There exists" becomes "for all" and predicate is negated
$P \rightarrow Q$	$P \wedge \neg Q$	Important for conditional statements
$P \wedge Q$	$\neg P \vee \neg Q$	De Morgan's Law
$P \vee Q$	$\neg P \wedge \neg Q$	De Morgan's Law

- **Statement:** $\exists x \in \mathbb{R}, x^2 + 1 = 0$
- **Negation Rule:**
- $\neg(\exists x, P(x)) \equiv \forall x, \neg P(x)$
 $\neg(\exists x \in \mathbb{R}, x^2 + 1 = 0) \equiv \forall x \in \mathbb{R}, x^2 + 1 \neq 0$
-  Simplified in words:
"For every real number x , $x^2 + 1$ is not zero."
- **Comment:** This is **true**, because $x^2 + 1 = 0$ has no real solution.

Negation of Nested Quantified Statements

Original Statement	Negation	Notes
$\forall x, P(x)$	$\exists x, \neg P(x)$	"For all" becomes "there exists" and predicate is negated
$\exists x, P(x)$	$\forall x, \neg P(x)$	"There exists" becomes "for all" and predicate is negated
$P \rightarrow Q$	$P \wedge \neg Q$	Important for conditional statements
$P \wedge Q$	$\neg P \vee \neg Q$	De Morgan's Law
$P \vee Q$	$\neg P \wedge \neg Q$	De Morgan's Law

- **Statement:** $\exists x \in \mathbb{Z}, \forall y \in \mathbb{Z}, x \cdot y = 0$
- **Step 1:** Negate outer $\exists$:
- $\neg(\exists x, \forall y, x \cdot y = 0) \equiv \forall x \in \mathbb{Z}, \neg(\forall y, x \cdot y = 0)$
- **Step 2:** Negate inner $\forall$:
- $\neg(\forall y, x \cdot y = 0) \equiv \exists y \in \mathbb{Z}, x \cdot y \neq 0$
-  **Final negation:**
- $\forall x \in \mathbb{Z}, \exists y \in \mathbb{Z}, x \cdot y \neq 0$
- **In words:**
"For every integer x , there exists an integer y such that $x \cdot y \neq 0$."
- **Comment:** False — if $x = 0$, then $x \cdot y = 0$ for all y .

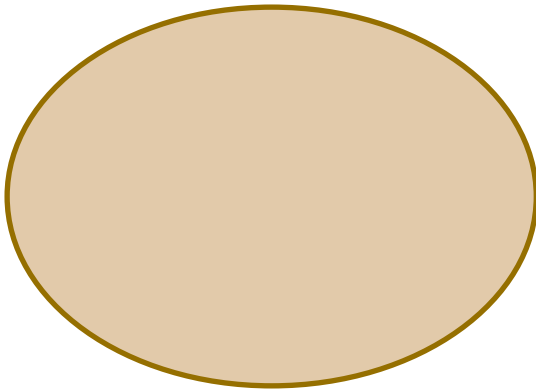


Computational Complexity

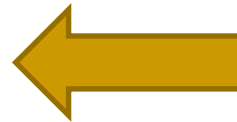
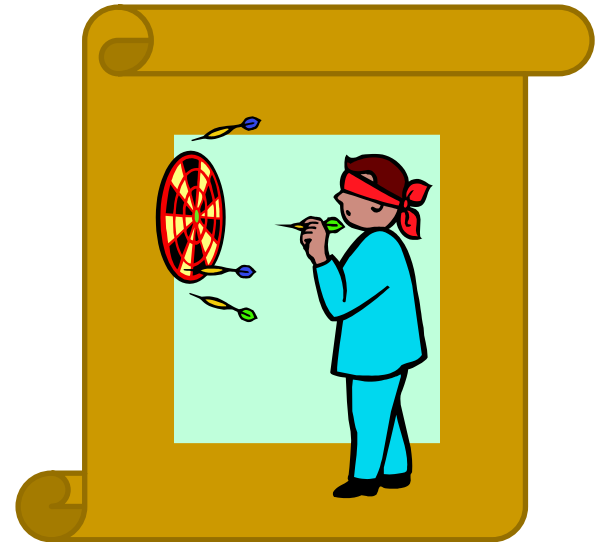


Complexities: Problem and Algorithm

Problem Complexity



Algorithm Complexity



NP-Complete Problems

- Digital design and synthesis is full of NP-complete problems
- Graph coloring
- Allocation/assignment
- Scheduling
- Graph partitioning
- Satisfiability (and 3SAT)
- Covering
- . . . and many more

Conjecture on Hardness of Problems

- There is a class of problems, NP-complete,
 - for which nobody has found polynomial time solutions
- It is possible to convert between these problems in polynomial time
 - Thus, if it is possible to solve any problem in NP-complete in polynomial time, all can be solved in polynomial time
- $P \subseteq NP$
- Unproven conjecture: $P \neq NP$

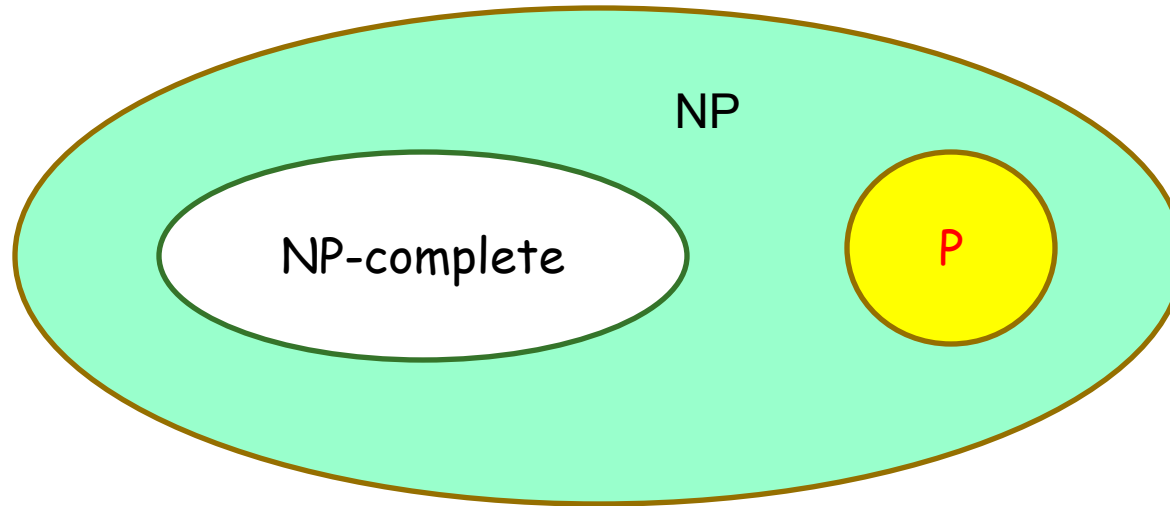
NP

- What is NP?
 - Nondeterministic polynomial time.
- A computer that can simultaneously follow multiple paths in a solution space exploration tree is nondeterministic.
 - Such a computer can solve NP problems in polynomial time.
- Nobody has been able to prove either
 - $P \neq NP$ or
 - $P = NP$

NP-completeness

- If we define NP-complete to be a set of problems in NP for which any problem's instance may be converted to an instance of another problem in NP-complete in polynomial time,
 - Then $P \subset NP \Rightarrow \text{NP-complete} \cap P = \emptyset$

Basic Complexity Classes



- P solvable in polynomial time by a computer (Turing Machine)
- NP solvable in polynomial time by a nondeterministic computer
- NP-complete converted to other NP-complete problems in polynomial time

References

- Discrete Structures, Logic, and Computability by James L. Hein
- https://en.wikipedia.org/wiki/Fibonacci_number
- [https://en.wikipedia.org/wiki/Recursion_\(computer_science\)](https://en.wikipedia.org/wiki/Recursion_(computer_science))